

PHOT 110: Introduction to programming

Lecture notes

Michaël Barbier

2026-04-20

Table of contents

Preface	4
1 Introduction	5
2 Python basics	6
2.1 Python program structure	6
2.2 The Python console	6
2.3 Python scripts	6
2.4 Data objects	7
2.4.1 Object type	7
2.4.2 Variables	8
2.5 Expressions & Operators	8
2.5.1 Structure of an expression	9
2.5.2 Operator precedence and brackets	9
2.6 Text objects	10
2.6.1 Operators working on text objects	10
2.6.2 Formatting strings and numbers	11
2.7 Python reserved words or keywords	11
2.8 input & output	11
2.9 Indentation	12
2.10 Import statements	12
2.11 Tidy and Documented code	13
2.11.1 Comment lines	13
2.11.2 Multi-line comments	13
2.11.3 Docstrings	13
2.12 Program Errors	14
2.12.1 Syntax error examples	14
2.12.2 Runtime error examples	15
3 Control flow: conditional branching and loops	16
3.1 Conditional branching: if/elif/else	16
3.2 While Loop	19
3.3 Python lists	20
3.3.1 INTERMEZZO: Methods versus Functions	21
3.3.2 Selecting elements in a list	21
3.3.3 Slicing a list	22
3.4 The for loop	23
3.4.1 Repeating a process n times	23
3.5 Example: The trajectory of a ball	25
4 Sequences: lists, ranges, tuples, strings	27
4.1 Mutable versus Immutable	27
4.1.1 Numbers are immutable	28
4.1.2 Strings are immutable	28
4.1.3 Lists are mutable	28
4.2 Tuples are immutable lists	30

4.3	Common sequence operations	31
4.4	Sequence operations: only mutable	32
4.5	Sequence operations: also for immutable types	33
4.6	String specific operations	33

Preface

This is a Quarto book.

To learn more about Quarto books visit <https://quarto.org/docs/books>.

1 Introduction

These are the lecture notes of the introduction to programming course given to 1th year photonics students at IZTECH. The course book being used is Hans Petter Langtangen (5th edition) See Knuth (1984) for additional discussion of literate programming.

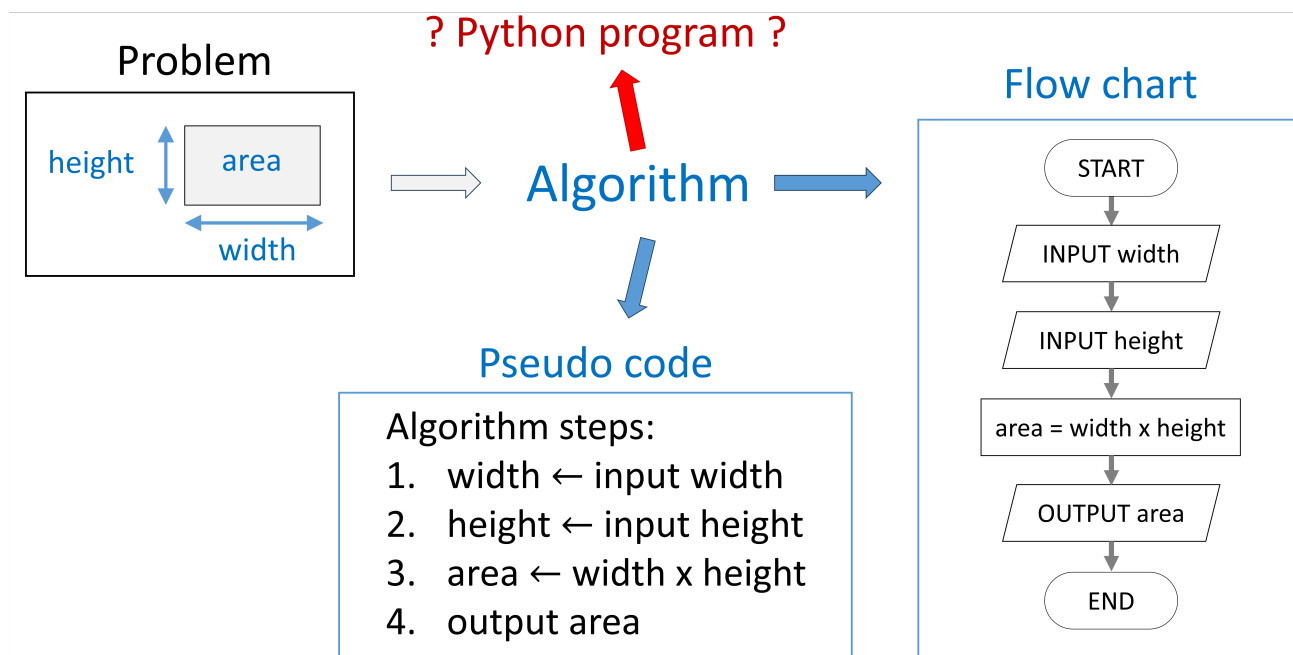
Hans Petter Langtangen <https://hplgit.github.io/>

<https://hplgit.github.io/scipro-primer/>

<https://hplgit.github.io/scipro-primer/slides/index.html>

Book on Numerical Methods

Computer science is the discipline which studies solving problems by designing and implementing algorithms executed on a computer. An algorithm is here a recipe or procedure containing simple steps to solve the problem and a stopping criterium which determines when the algorithm is finished and the solution found. When an algorithm is implemented on a computer in a computer language or programming language then we call the procedure a program. Here, we will provide the basics to solve scientific problems and implement numerical methods in **Python**. A large part is dedicated to describing the Python syntax, i.e. the grammar of this computer language.



2 Python basics

Python is an interpreted language, this means that the Python script is not first compiled to machine-code (an executable), but there is an interpreter (also a computer program) that executes the Python commands for us. This saves us the time and extra step to compile the script, but we need to have the Python interpreter installed and programs typically run slower. Later on, we will see that the last performance issue often can be mitigated by using highly optimized (and compiled) helper libraries for computationally heavy tasks, e.g. the Numpy and Scipy packages.

2.1 Python program structure

A Python script is a sequence of **statements** (algorithm steps), and **definitions** (functions, classes, ...)

- Definitions are evaluated, i.e. they can be used afterwards
- Statements are executed

In Python we can execute a program in two ways:

- Execute statements **one by one** in the **Python Console**, this is directly talking to the Python interpreter, or
- Statements **stored** as a sequence in a **script file**

2.2 The Python console

Execute statements one by one in the Python Console

```
1 >>> print(6)
2 6
3 >>> a = 10
4 >>>
```

Useful for:

- quick calculations
- testing the working of commands

2.3 Python scripts

Statements stored in a script file

```
a = 10
b = 25
prod = a * b
print(f"{a} times {b} = {prod}")
```

10 times 25 = 250

Useful for:

- running a program multiple times
- incorrect lines can easily be found and corrected
- larger programs

2.4 Data objects

In Python **everything is a data object**: numbers, text, functions, class instances, etc. Python **statements** perform actions on data objects.

Data objects can be scalar, of a basic data type, or composite (combining some of the basic data types). Some scalar data objects are listed in the table below:

Type	Description	(Example) values
bool	Boolean value	True, False
int	Integer numbers	.. , -2, -1, 0, -1, -2, ..
float	Floating point	3.56, 23e-3, 0.0079
NoneType	Indicates no object	None

2.4.1 Object type

Every object has a type, for example a decimal number has as type `float`:

```
print(type(6.22e23))
```

```
<class 'float'>
```

It is possible to convert between certain types, this we call **type casting**. For example, a logical true (`True`) corresponds to an integer with value 1. While the logical false (`False`) would be casted into 0.

```
it_is_raining = True
print(type(it_is_raining))
an_integer = int(it_is_raining)
print(type(an_integer))
```

```
<class 'bool'>
<class 'int'>
```

When doing specific operations between objects, objects can undergo automatic type casting. For example, when multiplying an integer with a decimal number, the product could become a decimal number, therefore the Python interpreter gives us the result as a decimal number (a `float`). When Python prints a decimal number it shows the decimal point, so we can understand what the result has as type. Some typical cases of automatic type casting are:

```
float <= int * float
float <= int + float
```

```
float <= int / int
```

We can see this in practice in following code:

```
a = 4
b = 0.357
print( f"Type of (a * b) = {type(a * b)}" )
print( f"Type of (a / b) = {type(a / b)}" )
print( f"Type of (a + b) = {type(a + b)}" )
print( f"Type of (a / 25) = {type(a / 25)}" )
```

```
Type of (a * b) = <class 'float'>
Type of (a / b) = <class 'float'>
Type of (a + b) = <class 'float'>
Type of (a / 25) = <class 'float'>
```

Here, the Python function `type(a)` gives the type of object with name `a`, and `type(a * b)` the type of the product `a` times `b`, etc.

2.4.2 Variables

To be able to reuse values and to operate on them we use variables, that is, we assign a name to them. A **variable** is a name bound to an **object**:

- The object is **assigned** to the variable,
- In pseudo code indicated by $\leftarrow$

In python the assignment operator is the “=” sign:

```
a_variable_name = 4.5
```

The variable can be re-assigned another value or object:

```
v = 4
v = v + 2                # v becomes 6
v = "Now v is assigned text"  # v has type str
```

The assignment operator is thus not the same as the “is equal to” operator from mathematics, this is especially clear when considering the statement `v = v + 2` above.

2.5 Expressions & Operators

Objects can be combined by **operators** in **expressions**. Typically we consider arithmetic operators and logical operators working on our basic object types (numbers and logical values), although we will see more complex operators later on.

Most used arithmetic operators:

symbol	description	example
**	Power	$3^{**}2 = 9$
/	Division	$5 / 4 = 1.25$
*	Multiplication	$3 * 4 = 12$
+	Addition	$5 + 7 = 12$
-	Subtraction	$6 - 9 = -3$
%	modulo	$34 \% 6 = 4$

Most used logic operators:

symbol	description	example
<, >	smaller/larger than	$5 > 4 \rightarrow \text{True}$
==	is equal to	$3 == 6 \rightarrow \text{False}$
<=, >=	smaller/larger or equal	$5 <= 5 \rightarrow \text{True}$
and	boolean AND	$\text{True and False} \rightarrow \text{False}$
or	boolean OR	$\text{True or False} \rightarrow \text{True}$
not	boolean NOT	$\text{not True} \rightarrow \text{False}$

2.5.1 Structure of an expression

An expression can be built using following rules recursively:

- $\langle \text{expression} \rangle \stackrel{\text{def}}{=} \langle \text{object} \rangle$ (an object is an expression)
- $\langle \text{expression} \rangle \stackrel{\text{def}}{=} \langle \text{operator} \rangle \langle \text{expression} \rangle$
- $\langle \text{expression} \rangle \stackrel{\text{def}}{=} \langle \text{expression} \rangle \langle \text{operator} \rangle \langle \text{expression} \rangle$

Expressions result into values and can be assigned to variables:

```
x = 4
y = 2*x**2 + 3*x - 5
print(f"The value of y = {y}")
```

The value of y = 39

2.5.2 Operator precedence and brackets

Precedence of operators is similar to mathematics: taking a power has priority over multiplication (division) and multiplication (division) has priority over addition (subtraction). For equal priority we apply the operators from left to right.

List of precedence of operators can be found on the python.org website: <https://docs.python.org/3/reference/expressions.html#operator-precedence>

Round brackets can be used to give priority or add clarity:

```
x = 2
y = (12 - x) / 10
print(f"The value of y = {y}")
```

The value of y = 1.0

2.6 Text objects

Text objects are called strings: the type is `str`. A string is defined by using single or double quotes:

```
"This is a string, single 'quotes' can be used here"  
'Here we can use double "quotes" inside it'
```

Strings can also cover multiple lines, for that we use triple quotes:

```
""" This string runs over multiple lines.  
Another line starts here.  
And another one.  
"""
```

Also triple single quotes work as well.

2.6.1 Operators working on text objects

Appending one string to another with the “+” operator:

```
a_verb = "flies"  
print("The balloon" + a_verb)  
print("The balloon" + " " + a_verb)
```

```
The balloonflies  
The balloon flies
```

Multiplying strings:

```
print(5 * "balloon ")
```

```
balloon balloon balloon balloon balloon
```

Casting a string to a number and vice versa:

```
print(5 * int("100"))  
print(10 * str(50))
```

```
500  
50505050505050505050
```

2.6.2 Formatting strings and numbers

A formatted string is a string with prefix `f` or `F`:

```
the_radius = 25
formatted_str = f"A circle with radius {the_radius} m"
print(formatted_str)
```

A circle with radius 25 m

More complex formatting:

```
a = 1/6; b = 0.0145; c = 23e-6
print(f"The product {a} x {b} = {a * b}")
# Use format {variable:No_space.No_sign_digits}
print(f"{a:8.2} x {b:8.2} = {a * b:8.2}")
print(f"{a:8.2} x {c:8.2} = {a * c:8.2}")
```

```
The product 0.16666666666666666 x 0.0145 = 0.0024166666666666667
    0.17 x    0.015 =    0.0024
    0.17 x  2.3e-05 =   3.8e-06
```

2.7 Python reserved words or keywords

- A list of words **reserved** for Python
- You cannot name variables (or functions/classes) similar

```
# List of keywords can be found in the keyword package
import keyword

print(keyword.kw_list)
```

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del', 'elif', 'except', 'exec', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'le', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
```

2.8 input & output

When not in interactive mode:

- `input()` can be used to ask user text input
- `print()` can be used to output text

```
1 # parameters
2 pizza_price = 200
3 extra_cheese_price = 20
4
5 # input accepts a string parameter
6 n_pizza = input("How many pizza's do you want: ")
7 extra_cheese = input("""Do you want extra cheese?\n
8     For yes press [1]\n
```

```

9     For no press [0]\n
10 Enter your choice"")
11
12 # Output to the user
13 print("The total cost is {int(n_pizza * (pizza_price + extra_cheese))}")

```

2.9 Indentation

- Is the amount of space preceding statements
- Indentation should be the same within a code block
- **Spaces** and **Tabs** are different
- Following code will give an `IndentationError: unexpected indent`

```

a = 4
b = 2
  c = 3
y = a * (b + c)

```

```

IndentationError: unexpected indent (3759259151.py, line 3)
Cell In[13], line 3
    c = 3
    ^
IndentationError: unexpected indent

```

2.10 Import statements

Python packages can be imported in multiple ways

```

1 # import the package with its name
2 import math
3
4 area = 3**2 * math.pi
5 print(f"Area of a circle with radius 3 = {area}")
6
7 # import functions of the package separately
8 from math import sin, cos, pi, sqrt
9
10 angle = 23/180*pi
11 formula = sqrt(2/3) * sin(angle) + cos(angle)
12
13 # import packages or functions and rename them
14 from math import sqrt as sr
15
16 print(f"The square root of 2 = {sr(2)}")

```

2.11 Tidy and Documented code

A tidy writing style has many benefits beyond just being neat. We can define it by adhering to the following points:

- Writing **readable** code
- Clear and not too long variable names
- Use spaces in a consistent manner
- Use a Python code formatter such as **Black**:
<https://github.com/psf/black>
- Document code both with **comments** and **docstrings**

2.11.1 Comment lines

- Text of a line after a hash symbol is ignored
- The hash symbol can be in the beginning of the line:

```
# A comment on a single line
```

Comments can explain the next line of code

```
# Convert Matlab-indices to zero-based  
ind = m_index - 1
```

Or the comment can start after a statement

```
y = 12 + x # An inline comment: a hash symbol after 2 spaces
```

2.11.2 Multi-line comments

Multi-line strings can be used as comments, but

- they are not meant as comments (not ignored)
- they can become docstrings (documentation-strings)

```
'''  
This multi-line string is not assigned to a  
variable, therefore doesn't influence your code  
directly  
'''
```

2.11.3 Docstrings

- Docstrings are multi-line strings providing documentation
- They populate the `__doc__` variable
- The `help()` function uses the `__doc__` variable

```
import math
print(math.__doc__)
```

This module provides access to the mathematical functions defined by the C standard.

```
import math
help(math.sin)
```

Help on built-in function sin in module math:

```
sin(x, /)
    Return the sine of x (measured in radians).
```

- We will revisit Docstrings for functions and classes later.

2.12 Program Errors

Different types of errors can be encountered

- **Syntax errors:** code inconsistent with the Python language, the code will not run (i.e. not start).
- **Runtime errors:** exceptions occurring while the program runs, the code will crash.
- **Semantic errors:** the code does not what was intended. This type of errors is the hardest to track.

2.12.1 Syntax error examples

- Usage of unknown identifiers (variable, function, class)

```
positive_number = uint(45)
```

NameError: name 'uint' is not defined

```
-----
NameError                                Traceback (most recent call last)
Cell In[16], line 1
----> 1 positive_number = uint(45)
NameError: name 'uint' is not defined
```

- Misaligned indentation

```
a = 34
b = 5
print(f"The sum is {a+b}")
```

```
IndentationError: unexpected indent (3752930843.py, line 2)
```

```
Cell In[17], line 2
```

```
b = 5
^
```

```
IndentationError: unexpected indent
```

- Mistaken symbol ":" instead of ";"

```
a = 3: b = 5
```

```
SyntaxError: invalid syntax (173182802.py, line 1)
```

```
Cell In[18], line 1
```

```
a = 3: b = 5
^
```

```
SyntaxError: invalid syntax
```

2.12.2 Runtime error examples

- Division by zero

```
fraction = 1 / 0
```

```
ZeroDivisionError: division by zero
```

```
-----
ZeroDivisionError                                Traceback (most recent call last)
```

```
Cell In[19], line 1
```

```
----> 1 fraction = 1 / 0
```

```
ZeroDivisionError: division by zero
```

- Type mismatch

```
netto_price = 230.50
```

```
kdv_ratio = 0.21
```

```
print("Price: " + netto_price + " + " + kdv_ratio + " kdv")
```

```
TypeError: can only concatenate str (not "float") to str
```

```
-----
TypeError                                Traceback (most recent call last)
```

```
Cell In[20], line 3
```

```
1 netto_price = 230.50
```

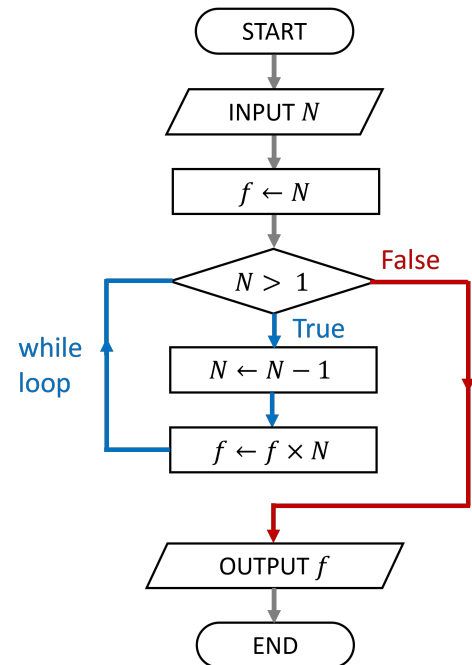
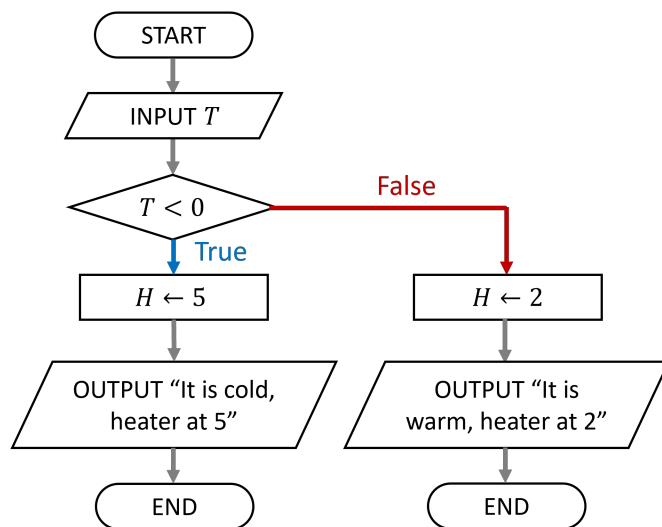
```
2 kdv_ratio = 0.21
```

```
----> 3 print("Price: " + netto_price + " + " + kdv_ratio + " kdv")
```

```
TypeError: can only concatenate str (not "float") to str
```

3 Control flow: conditional branching and loops

- Branching & loops



In computational algorithms the next steps taken can depend on certain conditions: this is called branching of the flow. Often this branching is designed such that a “loop” occurs in the control flow: in this situation a specific code-block will be repeated until some stopping condition (or for a finite amount of times). In Python there are specific constructs for this purpose:

- Branching if/else statements
- The while loop
- Lists of objects
- The for loop: iterating over a list

3.1 Conditional branching: if/elif/else

Run **code-blocks** according to a **condition**

- An **if** code-block is executed when the condition is **True**
- [Optional] An **else** code-block is executed when the condition is **False**

```
if <condition>:  
    <statement>  
    ...
```

```
if <condition>:  
    <statement>  
    ...  
else:
```

```
<statement>
...
```

If one wants to test for multiple conditions and act accordingly, the **elif** keyword acts as a **else if**. Multiple **elif** statements can follow an **if**, with an optional **else**:

```
if <condition>:
    <statement>
    ...
elif <condition>:
    <statement>
    ...
elif <condition>:
    <statement>
    ...
else:
    <statement>
    ...
```

```
if <condition>:
    <statement>
    ...
elif <condition>:
    <statement>
    ...
elif <condition>:
    <statement>
    ...
```

The statements that follow the **if** clause should be indented and form the **code-block** that will be executed according to a **condition**. An **if** code-block is executed when the condition is **True**, for example:

```
age = 46
if age >= 16:
    print("You can drive a tractor")  # if code-block
```

You can drive a tractor

The **indented code-block** can also contain multiple statements, which will be all executed when the condition is **True**:

```
speed_limit = 120
speed = 137
if speed > speed_limit:
    speed_diff = speed - speed_limit
    print(f"You drive {speed_diff} km/h too fast")
```

You drive 17 km/h too fast

When the condition is **False** the **if** code-block will not be executed. In case an **else** code-block is added it will be executed when the condition is **False**:

```
age = 11
if age > 18:
    print("You can drive a car")
else:
    print("You should take the bus")
```

You should take the bus

In case you want to cover more than two cases, i.e. you have multiple conditions, the **elif** keyword can be used. You can use it multiple times, and optionally still use an **else** clause in the end:

```
age = 17
if age > 18:
    print("You can drive a car")
elif age > 16:
    print("You can drive a tractor")
elif age > 15:
    print("You can drive a moped")
else:
    print("You can ride a bicycle")
```

You can drive a tractor

In summary we can run **code-blocks** according to a **condition**

- **if** code-block is executed when the condition is **True**
- **else** code-block is executed when the condition is **False**
- **elif** keyword acts as a **else if**
- The **indented code-blocks** can contain multiple statements
- The indentation should be the same within each code-block

Be careful with the indentation of the code-blocks otherwise your program will either exhibit unwanted behavior or will give an error, such as the following code:

```
age = 19
if age > 18:
    print("You can drive a car")      # This line is indented differently
    print("You can drive a bicycle")
    print("You can drive a tractor")
```

```
IndentationError: unindent does not match any outer indentation level (<string>, line 4)
File <string>:4
    print("You can drive a bicycle")
    ^
```

```
IndentationError: unindent does not match any outer indentation level
```

3.2 While Loop

A **while loop** repeats a code-block until the **condition** is **False**

```
while <condition>:
    <statement>
    <statement>
    <statement>
    ...
    <statement>
```

A **while loop** is used when:

- we don't know how many iterations we need, and
- we have a stopping criterium/condition

As an example we update a certain number $x \rightarrow x^2 - 3x$ in each iteration and stop if the number becomes larger or equal to 1000:

```
x = 1
while x < 1000:
    x = x ** 2 - 3 * x
    print(x)
print("End of the program")
```

```
-2
10
70
4690
End of the program
```

Another a bit more complex example, we want to calculate the cumulative sums of the series $\sum_{n=1}^{\infty} \frac{1}{n^4}$ up to certain order N :

```
x = 0; dx = 1; dx_min = 0.001; n = 1
while dx > dx_min:
    dx = 1 / n ** 4
    x = x + dx
    print(f"The series up to order N = {n} is: {x:6.5}")
    n = n + 1
print("End of the program")
```

```
The series up to order N = 1 is:    1.0
The series up to order N = 2 is: 1.0625
The series up to order N = 3 is: 1.0748
The series up to order N = 4 is: 1.0788
The series up to order N = 5 is: 1.0804
The series up to order N = 6 is: 1.0811
End of the program
```

Notice that the while loop can get in an infinite loop! This can happen if the stopping criterium is not adequate.

- Stop the program with the stop button, in a terminal press key combination **Ctrl + c**
- Adapt the stopping criterium/condition

Try the following code to get in an infinite loop:

```
n = 0
while n > -100:
    n = n + 1
    print(f"The current number is: {n}")
```

The most frequently used loop structure is actually not the **while** loop but the **for** loop. The **for** loop has the advantage that it can be run a specific number of times. But before we go to the **for** loop we will have a brief look at the **list** data type, since this is often used in conjunction with **for** loops when iterating over the elements of a **list**.

3.3 Python lists

A Python list is a collection of objects with a certain order:

- A list can contain several objects
- The object types can be different
- Lists are also objects themselves

```
# A list with mixed object types
my_list_of_objects = ["It's Monday", False, 34, 23.4]

# Lists can be elements of a list
a_list_with_a_list = [5, 10.5, ["green", "red"], True]
print(a_list_with_a_list)
```

```
[5, 10.5, ['green', 'red'], True]
```

The **length of the list** is the number of elements. Applying the **len()** function: **len(a_list)**, allows us to find the number of elements.

```
number_of_objects = len(my_list_of_objects)
print(number_of_objects)
```

```
4
```

There are many methods defined on lists, such as: Appending an element, removing an element, inserting an element, etc.

When we **append** an element, the number of elements of the list increases with one:

```
a_list = ["First", False, 34, 23.4]
print(a_list)
print(f"The length of the list = {len(a_list)}")
a_list.append("extra_element")
print(a_list)
print(f"The length of the adapted list = {len(a_list)}")
```

```
['First', False, 34, 23.4]
The length of the list = 4
['First', False, 34, 23.4, 'extra_element']
The length of the adapted list = 5
```

3.3.1 INTERMEZZO: Methods versus Functions

Notice that we use the dot-notation: `the_list.append(the_element)`. We already saw this notation in a different context: when using for example the square root function `sqrt()` of the `math` module:

```
import math
a = math.sqrt(2) # This is a function
print(f"The square root of 2 = {a}")
```

The square root of 2 = 1.4142135623730951

However, here this notation is not used to use a function of an imported package/module, here we use it to call a method on an object: we use the method `append()` on the list (the object).

We will see methods afterwards in more detail, but let us for now summarise. We already saw multiple functions like: `print()`, `input()`, `len()`, etc. Methods are similar to functions but they are bound to an object. They can change the object on which they work (e.g. like `append()`) but don't have to. The manner in which we use a method is:

```
a_list = ["First", False, 34, 5, 34] # Define the list
a_list.remove(34) # Remove first 34
a_list.insert(3, "inserted_string") # Insert str
print(a_list) # Print the list
```

```
['First', False, 5, 'inserted_string', 34]
```

There are more methods we can make use of, see <https://docs.python.org/3/tutorial/datastructures.html#more-on-lists>.

- We will see how to make our own methods (and classes) later in the chapter on object oriented programming

3.3.2 Selecting elements in a list

We can select an element of a list by its index, this is, its place in the list.

- syntax for indexing: `a_list[element_index]`
- index is zero-based
- negative index starts from the end of the list

Using the above rules we can select a specific element from the list, when using index 0 we will get the first element:

```
# Printing the first and then the second element
a_list = ["First", False, 34, 23.4]
print(a_list[0])
print(a_list[1])
```

```
First
False
```

Or selecting the last element using negative indexing:

```
# Using negative indexing
a_list = ["First", False, 34, 23.4]
print(a_list[-1])
```

```
23.4
```

3.3.3 Slicing a list

We can also select multiple elements, this is called **slicing**, the syntax for slicing is: `a_list[start:stop_exclusive]` where we employ the colon-operator to separate the start and stop indices.

```
# A list with mixed object types
a_list = [23, 45, 65, 78, 92, 100, 102, 105 ]
print(a_list[2:5])
```

```
[65, 78, 92]
```

The **start** index can be omitted, then the selection is from the first element with index 0. For example, we can select the first five elements:

```
# An empty start_index starts from the first index
a_list = [23, 45, 65, 78, 92, 100, 102, 105 ]
print(a_list[:5])
```

```
[23, 45, 65, 78, 92]
```

Likewise when the **stop** index is omitted, the selection is until (and including) the last element. For example, selecting all elements from the fourth element on can be done as:

```
# An empty end_index end at the last index
a_list = [23, 45, 65, 78, 92, 100, 102, 105 ]
print(a_list[3:])
```

```
[78, 92, 100, 102, 105]
```

An additional **step** parameter can be used to skip elements: `a_list[start:stop_exclusive:step]`, again we use the colon-operator to separate the start/stop/step parameters. We can for example select every second element:

```
# Take every second element by stepping
a_list = [23, 45, 65, 78, 92, 100, 102, 105 ]
print(a_list[1::2])
```

```
[45, 78, 100, 105]
```

Or we can **reverse** the list by using a negative step:

```
# Reverse the list by negative step
a_list = [23, 45, 65, 78, 92, 100, 102, 105 ]
print(a_list[::-1])
```

```
[105, 102, 100, 92, 78, 65, 45, 23]
```

3.4 The for loop

Similar to the while loop, the **for** loop can be used to repeat a process. Unlike the while loop the for loop is meant for when either:

- the number of **iterations** is known, or
- we iterate over a list of elements

The structure of the for loop to iterate over the elements of a list is as follows:

```
for <element> in <list>:
    <statement>
    <statement>
    ...
    <statement>
```

For example, to iterate over the working days of the week and print them, we could use

```
# Print all elements of a list
days = ["Mon", "Tue", "Wed", "Thu", "Fri"]
for el in days:
    print(el)
```

```
Mon
Tue
Wed
Thu
Fri
```

3.4.1 Repeating a process n times

The for loop can also be used to repeat a process a known number of times. For this we replace the list by the **range** function, which can represent a “list” of numbers. The **range** function returns actually an object of type **range** (which belongs to the more general class of **sequence** just like **list**) for integer numbers, we can construct it as follows: **range(start, stop_exclusive, step)**, where:

- **start** is the first integer (default is zero),
- **stop_exclusive** is the last integer,
- **step** (default value is one),
- See also: <https://docs.python.org/3/library/stdtypes.html#range>

The **range()** does not give a list of integers, but a **range** object: this object is more memory efficient, it only generates the integer numbers when asked for (imagine you want to do billions of iterations). We can cast a range object into a list though:

```
a_range = range(1, 10, 2)  # Construct a range
print(a_range)             # Lazy evaluated
print(list(a_range))       # Converted to a list
```

```
range(1, 10, 2)
[1, 3, 5, 7, 9]
```

We can use a range object directly in a for loop instead of the list to iterate over:

```
# Use the range function to get a sequence of numbers
for i in range(1,10,2):
    print(i)
```

```
1
3
5
7
9
```

Using range we can also get the indices of a list. We can use the indices to print only the odd days of the week for example:

```
# Use the range function to get indices
days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
for i in range(0,len(days),2):
    print(days[i])
```

```
Mon
Wed
Fri
Sun
```

Or one could apply the same index to multiple lists:

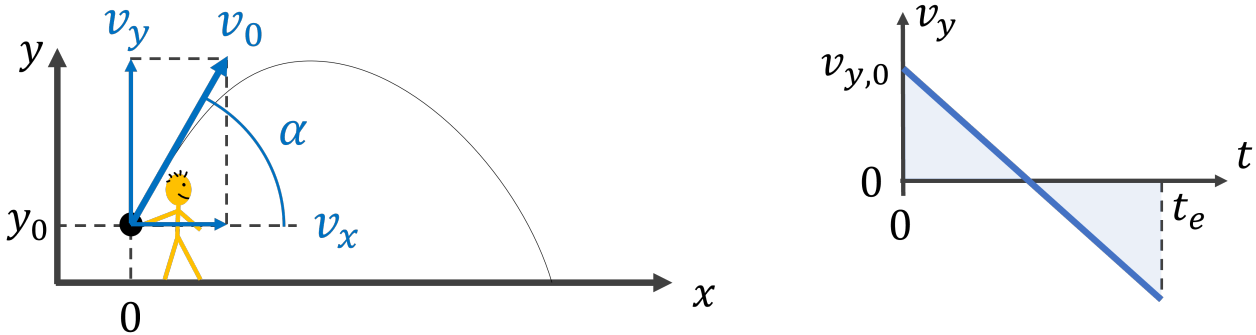
```
# Use the range function to get indices and apply to 2 lists
names = ["Mehmet", "Ceren", "Nisan", "Derin", "Ahmet", "Ali"]
lengths = ["189", "170", "182", "178", "177", "180"]
for i in range(len(names)):
    print(f"{names[i]} is {lengths[i]} cm.")
```

```
Mehmet is 189 cm.
Ceren is 170 cm.
Nisan is 182 cm.
Derin is 178 cm.
Ahmet is 177 cm.
Ali is 180 cm.
```

Notice that both lists need to have the same length and the elements should be corresponding to make sense.

3.5 Example: The trajectory of a ball

In science and engineering we often use numerical methods to simulate systems. As a practical example of using a while loop, let us calculate the trajectory $(x(t), y(t))$ of a ball that is thrown numerically. The ball has an initial position $(x(0), y(0)) = (0, 1.20)$ (in meters) and velocity $v_0 = 6$ m/s thrown under an angle of 37 degrees, see the figure below.



We consider following conditions:

- Gravitation: $g = 9.81$ m/s²
- Air resistance can be ignored for a “slow” heavy ball
- The horizontal velocity is constant

The numerical algorithm calculates the balls position at small discrete time steps and updates the ball's velocity at each time step. The resulting coordinates form an approximate trajectory. The code uses the while loop to calculate subsequent coordinates until the ball drops on the ground (the stopping criterium). See the code below.

```
1  # Load library for sine and cosine
2  import math
3
4  # Algorithm parameters in MKS units
5  v0 = 6
6  angle_in_degrees = 37
7  g = 9.81
8  x = 0; y = 1.20; t = 0
9
10 # Calculate initial velocity in x
11 #    and y directions
12 angle = angle_in_degrees * math.pi/180
13 vx = v0 * math.cos(angle)
14 vy = v0 * math.sin(angle)
15
16 # Initiate values at time t = 0
17 x_values = list([x])
18 y_values = list([y])
19 t_values = list([t])
20
21 dt = 0.1
22 while (y >= 0) and (t < 10):
23     # update velocities (vx, vy), coords
24     #    (x, y), and time t
25     vy = vy - g*dt
```

```

26  y = y + vy*dt
27  x = x + vx*dt
28  t = t + dt
29  x_values.append(x)
30  y_values.append(y)
31  t_values.append(t)
32  print(f't = {t:.2f}: ({x:.2f},{y:.2f})')

```

```

t = 0.10: (0.48,1.46)
t = 0.20: (0.96,1.63)
t = 0.30: (1.44,1.69)
t = 0.40: (1.92,1.66)
t = 0.50: (2.40,1.53)
t = 0.60: (2.88,1.31)
t = 0.70: (3.35,0.98)
t = 0.80: (3.83,0.56)
t = 0.90: (4.31,0.04)
t = 1.00: (4.79,-0.58)

```

4 Sequences: lists, ranges, tuples, strings

We already briefly saw lists, ranges, and strings before, these data types fall actually under a common denominator: the sequence. Sequences are **collections** of elements with an **order**. There are also **unordered collections** in Python: **sets**. The following types are typical sequences:

- **list**: adaptable sequence of objects,
- **range**: sequence of integers,
- strings (**str**): fixed sequence of characters,
- **tuple**: a fixed sequence of objects, i.e. like a list but not adaptable,
- ...

It is useful to know that these are all sequences because sequences have common operations: indexing, slicing, certain functions such as `len()`, and methods such as `.sort()`.

We can split sequences in two groups: those that are mutable (adaptable) and those that are immutable (fixed). Mutability is actually an important concept and determines which of the above mentioned common operations can be applied. Before we look at common operations of sequences, let us have a closer look at mutability.

4.1 Mutable versus Immutable

To understand what mutable means we first should appreciate that every object has a **unique ID** (in CPython this ID would be directly related to the address in memory). We can ask the ID of any Python object using the function `id()`, for example, an integer is an object and would have an ID:

```
i = 45
print(id(i))
```

4460262496

Other objects also have ID's:

```
a_string = "Pretzel"
a_list = [3, 105, 56]
a_range = range(4)
print(f"The ID of a_string: {a_string} = { id(a_string) }")
print(f"The ID of a_list: {a_list} = { id(a_list) }")
print(f"The ID of a_range: {a_range} = { id(a_range) }")
```

```
The ID of a_string: Pretzel = 4495242320
The ID of a_list: [3, 105, 56] = 4495428352
The ID of a_range: range(0, 4) = 4495241264
```

If an object is mutable, its content can change without changing its ID. In case an object is of immutable type, then the ID should change if the content changes. Let us take an integer number as example below.

4.1.1 Numbers are immutable

Since numbers are immutable in Python, their ID should change if you change them. Even if you assign them again to the same variable name! We can easily check this with the `id()` function:

```
i = 45
print(id(i))
i = i + 1
print(id(i))
```

```
4460262496
4460262528
```

4.1.2 Strings are immutable

Strings (`str`) are immutable just as numbers. A typical consequence is that operations on strings will provide a copy, the original string remains untouched:

```
A = "Pretzel"
B = A.upper()
print(f"String A: {A} is still the same as A: {A}")
print(f"String B: {B} is the uppercase version of A: {A}")
print(f"ID of B: {id(B)} is not equal to ID of A: {id(A)}")
```

```
String A: Pretzel is still the same as A: Pretzel
String B: PRETZEL is the uppercase version of A: Pretzel
ID of B: 4495239248 is not equal to ID of A: 4495242320
```

However, if you assign the new adapted string to the old variable name: `A = A.upper()`, then you will not have access to the initial string anymore.

4.1.3 Lists are mutable

Lists are mutable, their elements can change, added, removed, etc. Although the length of the list can change, the object will stay the same:

```
a_list = [3, "car", 5]
print(a_list)
print(id(a_list))

# Change 2nd element
a_list[1] = "tree"
print(a_list)
print(id(a_list))
```

```
[3, 'car', 5]
4495428544
[3, 'tree', 5]
4495428544
```

This has a consequence that whenever we adapt a list, we possibly lose the original elements. As an illustration of such side-effects, look at the following script, that calculates the median of a list of numbers.

```
# Input list of numbers
numbers = [3, 4, 5, 2, 8, 1, 6, 7]

# Algorithm to find the median
numbers.sort()
n = len(numbers)
if (n % 2) == 1:
    m = numbers[int((n + 1) / 2)]
else:
    m = (numbers[int(n/2-1)] + numbers[int(n/2)]) / 2

# Output the results
print(f"The median of {numbers} is {m}")
```

The median of [1, 2, 3, 4, 5, 6, 7, 8] is 4.5

Notice that the list is sorted after applying our approach to obtain the median, and we lost the original order of the list. This is a side-effect. We actually could avoid this by using the function `sorted()` that returns a copy instead of the method `sort()` that sorts the elements in-place:

```
numbers_1 = [3, 4, 5, 2, 8, 1, 6, 7]
sorted_numbers = sorted(numbers_1)
print(f"sorted_numbers: {sorted_numbers}, but numbers_1: {numbers_1} has still original order")
numbers_2 = [3, 4, 5, 2, 8, 1, 6, 7]
numbers_2.sort()
print(f"numbers_2 is sorted in place: {numbers_2}")
```

sorted_numbers: [1, 2, 3, 4, 5, 6, 7, 8], but numbers_1: [3, 4, 5, 2, 8, 1, 6, 7] has still original order
numbers_2 is sorted in place: [1, 2, 3, 4, 5, 6, 7, 8]

Let's rewrite our little median algorithm using the `sorted()` function (this is not a method but a function):

```
# Input list of numbers
numbers = [3, 4, 5, 2, 8, 1, 6, 7]

# Algorithm to find the median
sorted_numbers = sorted(numbers) # This line changed
n = len(numbers)
if (n % 2) == 1:
    m = sorted_numbers[int((n + 1) / 2)]
else:
    m = (sorted_numbers[int(n/2-1)] + sorted_numbers[int(n/2)]) / 2

# Output the results
print(f"The median of {numbers} is {m}")
```

The median of [3, 4, 5, 2, 8, 1, 6, 7] is 5.0

There are other methods that work “in-place” such as `s.reverse()` (reverses the order of elements of sequence `s` in-place), which have an alternative function: `reversed(s)`, that return a changed copy instead. This allows us to choose to either return a copy, albeit with possible large memory overhead, or do the operation in-place thereby losing the initial elements. Whatever approach you prefer, it is important to know whether a method/function **returns** an adapted copy of the object or it **adapts the object in-place**.

Nevertheless, with a list you cannot be sure that it didn’t get adapted without knowing what each step of the code exactly does.

4.2 Tuples are immutable lists

In case you want to make sure that your elements don’t change, you can use a **tuple** instead of a **list**. Tuples are similar to lists in everything except that you cannot change them. The syntax to create a tuple is also very similar, but round brackets (and) are used instead of square brackets [and]:

```
t = (3, "leaf", False)
print(t)
```

```
(3, 'leaf', False)
```

We can also cast from another collection to tuple, e.g. from `list`:

```
t = tuple([1, 3.23, 3, 5 + 6j])
print(t)
```

```
(1, 3.23, 3, (5+6j))
```

Tuples are immutable, trying to adapt them will lead to an error:

```
t = tuple([1, 3.23, 3, 5 + 6j])
t[0] = 2
```

```
TypeError: 'tuple' object does not support item assignment
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[11], line 2
      1 t = tuple([1, 3.23, 3, 5 + 6j])
----> 2 t[0] = 2
TypeError: 'tuple' object does not support item assignment
```

While methods such as `append()`, `remove()`, etc. will work on lists, they will not work on tuples.

So far we have seen 4 sequence types, where list is the only one which is mutable, list and tuple can both contain elements of different (mixed) types, see the table below.

Type	mutable	item type
<code>list</code>	yes	mixed types
<code>tuple</code>	no	mixed types
<code>range</code>	no	integers
<code>str</code>	no	characters

Type	mutable	item type
------	---------	-----------

Sequence operations are available according to mutability and type.

4.3 Common sequence operations

There are many sequence operations which can be applied to any type of sequence: lists, tuples, ranges, strings, etc. For example, verifying whether elements can be found in the sequence or not, or combining sequences by concatenation.

```

1 <bool> = el in s           # --> True/False
2 <bool> = el not in s       # --> True/False
3 s = s1 + s2               # concatenate s1 and s2
4 s * n or n * s           # n times concatenation

```

Some examples are:

```

print("a" in "Hallo")       # --> True/False
print(9 not in range(4, 10)) # --> True/False
print([100, 200] + [500, 20]) # concatenate s1 and s2
print(("cat", "dog") * 3)    # n times concatenation

```

```

True
False
[100, 200, 500, 20]
('cat', 'dog', 'cat', 'dog', 'cat', 'dog')

```

...

Notice that as an exception concatenation does not work for ranges, this is due to their lazy evaluation.

All sequences also allow you to select elements by indexing or slicing, and we can count or find specific elements.

```

el = s[i]           # Select element i
s = s[start:end+1:step] # Slicing
n = s.count[<el>]    # Count elements
i = s.index[<el>]    # index first el

```

For example for a tuple:

```

s = ("Malta", "Corsica", "Lesvos", "Malta"); print(s)
print(f"Third element of s: { s[2] }")
print(f"Slice of s: { s[1:] }")
print(f"'Malta' appears: { s.count('Malta') } times")
print(f"First index of 'Malta': { s.index('Malta') }")

```

```
('Malta', 'Corsica', 'Lesvos', 'Malta')
Third element of s: Lesvos
Slice of s: ('Corsica', 'Lesvos', 'Malta')
'Malta' appears: 2 times
First index of 'Malta': 0
```

But exactly the same would work for a list:

```
s = ["Malta", "Corsica", "Lesvos", "Malta"]; print(s)
print(f"Third element of s: { s[2] }")
print(f"Slice of s: { s[1:] }")
print(f"'Malta' appears: { s.count('Malta') } times")
print(f"First index of 'Malta': { s.index('Malta') }")
```

```
['Malta', 'Corsica', 'Lesvos', 'Malta']
Third element of s: Lesvos
Slice of s: ['Corsica', 'Lesvos', 'Malta']
'Malta' appears: 2 times
First index of 'Malta': 0
```

And also for a string where its elements are the characters composing the string:

```
s = "Malta and Corsica"; print(s)
print(f"Third element of s: { s[2] }")
print(f"Slice of s: { s[1:] }")
print(f"'a' appears: { s.count('a') } times")
print(f"First index of 'a': { s.index('a') }")
```

```
Malta and Corsica
Third element of s: l
Slice of s: alta and Corsica
'a' appears: 4 times
First index of 'a': 1
```

4.4 Sequence operations: only mutable

List is the only mutable sequence so far and we saw already many of possible operations:

```
<list>[i] = <el>
<list>.remove(<el>)           # remove first <el>
<list>.insert(i, <el>)        # insert <el> at index i
<el> = <list>.pop(i)          # return <el> at i and remove
<list>.append(<el>)           # Or: <list> += <el>
<list>.extend(<iterable>)     # Or: <list> += <iterable>
<list>.sort()                 # Sorts list in-place
<list>.reverse()              # Reverses list in-place
```

Examples of applying these operations on lists:

```

fruits = ["banana", "orange", "pear", "peach"]
nuts = ("almond", "walnut")

fruits.remove("banana")    # remove first <el>
fruits.insert(0, "mango")  # insert <el> at index i
el = fruits.pop(3)         # return <el> at i and remove
print(f"We popped {el}")
fruits.append("mandarin")  # Or: <list> += <el>
fruits.extend(nuts)        # Or: <list> += <iterable>
fruits.sort()              # Sorts list in-place
fruits.reverse()           # Reverses list in-place
print(fruits)

```

We popped peach
['walnut', 'pear', 'orange', 'mango', 'mandarin', 'almond']

4.5 Sequence operations: also for immutable types

Items of immutable types cannot change $\Rightarrow$ return value

```

<list> = sorted(<collection>)  # Returns sorted
<iter> = reversed(<list>)     # Returns reversed

```

Examples:

```

t = (3, 5, 7, 2, 1)
t_sort = sorted(t)    # Returns sorted
t_rev = reversed(t)   # Returns reversed
print(t)
print(t_sort)
print(list(t_rev))

```

```

(3, 5, 7, 2, 1)
[1, 2, 3, 5, 7]
[1, 2, 7, 5, 3]

```

4.6 String specific operations

Strings are also sequences, but since characters and text play such an important role, the `str` data type has some specific methods on top of the common sequence operations to fulfill specific needs. Below we list some often used string specific methods.

Often in data tables we want to avoid extra spaces around the data, one can use the method `strip()` to remove unwanted surrounding characters (most often extra space/tab characters):

```

print("  Some_extra spaces ")
print("  Some_extra spaces ".strip())
print("_Some text...".strip('.'))

```

```
Some_extra spaces
Some_extra spaces
_Some text
```

Splitting of a string in multiple parts or combining strings. For example this could be splitting a sentence in words. When combining (concatenating) strings one could think of adding the extension to a file name:

```
# Split and join strings
print("Split_on_underscore".split(sep="_"))
print( ".".join(["program", "exe"]) )
```

```
['Split', 'on', 'underscore']
program.exe
```

Another very common operation would be to search for a specific substring (word or letter combination) within a larger string. Think of the “Find & Replace” functionality in typical text editors:

```
print("Ha" in "Hallo")           # Verify whether "Ha" can be found
print("Ball".startswith("Bar"))  # Verify whether the string starts with "Bar"
print("Bicycle".find("cyc"))      # Returns the start-index of substring "cyc"
print("Bicycle".index("cyc"))     # Same as find, returns ValueError upon failure
```

```
True
False
2
2
```

Replacing words is another very common operation:

```
print("I like oranges".replace("oranges", "apples"))
```

```
I like apples
```

Other convenient methods are those which change the capitalization of characters. This can be for example useful to perform case-insensitive comparison of words:

```
print("GOOD morning".lower())
print("GOOD morning".upper())
print("GOOD morning".capitalize())
print("GOOD morning".title())
```

```
good morning
GOOD MORNING
Good morning
Good Morning
```

Lastly, each unicode character corresponds to a unique integer, translation between both can be of particular convenience when performing low-level interactions with the keyboard:

```
print(chr(68)) # Convert int to Unicode character.  
print(ord("A")) # Convert Unicode character to int.
```

D
65

There are more methods specific to strings, here we just listed the most used ones to give an impression of the possibilities.

Knuth, Donald E. 1984. "Literate Programming." *Comput. J.* 27 (2): 97–111. <https://doi.org/10.1093/comjnl/27.2.97>.